Agentic Lagrangian Extraction from the Literature

Midterm Progress Report & Technical Design

Program	Google Summer of Code 2026 | ML4SCI | HEPSIM5
Contributor	Ken Wu
Mentors	S. Mrenna (Fermilab), K. Matchev, A. Roman, T. Menzo (Alabama/Fermilab), I. Pang (Rutgers)
Framework	HEPTAPOD (`github.com/tonymenzo/heptapod`) | 350 hours, Advanced

---

1 Executive summary

The project automates a manual high-energy-physics bottleneck: turning a Beyond-Standard-Model (BSM) scenario into a validated FeynRules `.fr` simulation model. Given a scenario, an agent searches the literature, extracts the Lagrangian and its conventions, generates a syntactically correct `.fr` file, and validates it.

The central design choice is to be **harness-agnostic by construction**: we did not build an orchestrator. We built a layer of deterministic, schema-validated *tools*, exposed via the Model Context Protocol (MCP), plus a workflow *prompt*. The "brain" that sequences the tools, reads their errors, and runs the repair loop is a general agent harness—Claude Code, Codex, Cursor, or the Orchestral runtime—and it is **pluggable** (Section 3.1). This keeps the reasoning where harnesses already excel and keeps the physics deterministic and reproducible in tools.

At midterm, the front half of the pipeline is built and tested as HEPTAPOD tools: literature retrieval, schema-constrained extraction, and deterministic `.fr` generation, plus the workflow prompt and a benchmark scaffold (Section 2). The remaining half—physics-level validation, benchmarking against reference implementations, and extraction robustness—is scoped for the second half (Section 6). This report also folds in an adversarial self-review that reshaped several choices (Sections 4–5).

2 Progress against the project spec

Honest status against the six task ideas and four expected results.

Spec item	Status	Notes
1. Survey FeynRules models, catalog conventions	**gap**	Design informed by round-tripping one reference model; a catalog *artifact* (parsed from the FeynRules DB) is now the first second-half task.
2. Tool modules: literature search, extraction, `.fr` generation	**done**	`ArxivSearchTool`, `FetchPaperPDFTool`, `ExtractPaperTextTool`; `ExtractLagrangianTool`; `FeynRulesModel` schema + `GenerateFeynRulesModelTool`. INSPIRE reused. Wired into `toolkit.yaml`; tests pass.
3. Agent workflow: scenario → validated model	**partial**	Workflow prompt + harness-run repair loop + Orchestral demo exist; one end-to-end live run + committed artifacts still to do.
4. Validation: spectrum, gauge invariance, widths/xsec	**partial**	`ValidateModelTool` compiles `.fr`→UFO + checks files/particles. Gauge-invariance and width/cross-section reproduction are second-half (Section 5).
5. Test vs benchmark models in the FeynRules DB	**partial**	Eval scores extraction vs. hand-authored expectations; diffing against the reference `.fr`/UFO (SLQrules, 2HDM, DMsimp) is the planned upgrade.
6. Audit trail	**partial**	Detailed protocol in the workflow prompt; making it tool-emitted (`audit.json`) is a small task.

3 Architecture: a harness-agnostic tool layer

3.1 The harness boundary — what is swappable

The system separates cleanly into three layers. Everything that is *reasoning*—deciding the next tool, reading a FeynRules error, running the repair loop—belongs to a swappable agent harness. Everything that is a *deterministic capability*—search, `.fr` generation, compilation and validation—stays a fixed tool. External physics software is reused unchanged.

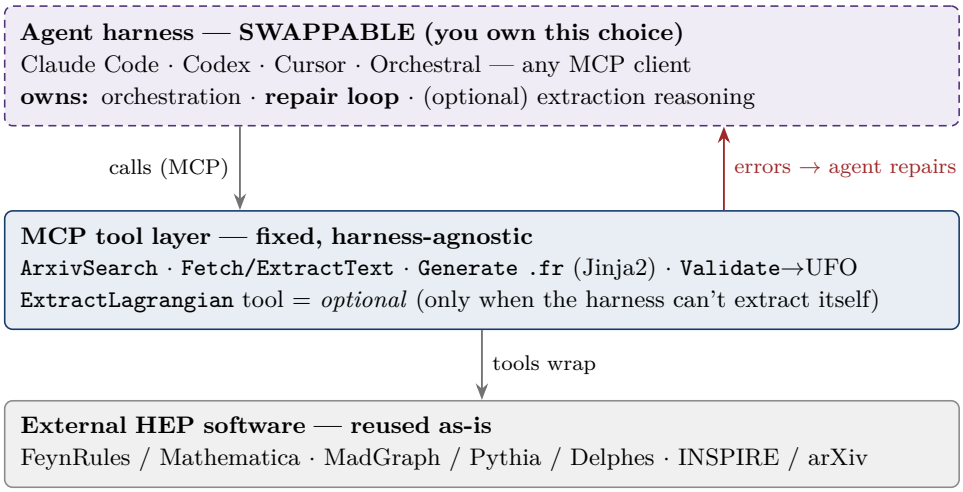

Figure 1: The swap boundary. The **harness** (dashed, top) is interchangeable and owns all reasoning, including the repair loop; the **MCP tool layer** is built once and reused by any harness; external software is untouched. Because there is no bespoke orchestrator, swapping Orchestral for Claude Code or Codex is a configuration change, not a rewrite.

The repair loop is the harness's job, and already is. There is no hand-written repair control-flow. The loop is specified in the workflow prompt, and whichever harness runs that prompt executes it: generate `.fr` → validate → read the concrete error → fix the model → regenerate. The tools exist to feed that loop actionable, structured errors: `GenerateFeynRulesModelTool` returns Pydantic validation failures, `ValidateModelTool` returns `{passed, checks[], feynrules_log}`. This is exactly the edit →run →read-failure →fix loop coding agents are built for, so Claude Code / Codex perform it natively.

One tool is deliberately optional: `ExtractLagrangianTool`. It embeds its own model call (schema-constrained decoding). When a capable coding-agent harness drives the pipeline, that is redundant—the harness reads the extracted paper text and emits the `FeynRulesModel` JSON itself. We keep the tool only for the headless/batch path (e.g. scoring many papers with a *pinned* model for reproducible benchmark numbers).

Two operating modes, same tools. Interactive / demo / midterm work is driven by **Claude Code or Codex** (strongest models, native repair loop, fully visible). Reproducible benchmarking uses the **Orchestral / thin owned loop** with a pinned model and logged traces, so eval numbers reproduce. This is the "harness-as-dev-tool vs. harness-as-reproducible-runtime" distinction, realized on one MCP tool layer.

3.2 End-to-end workflow

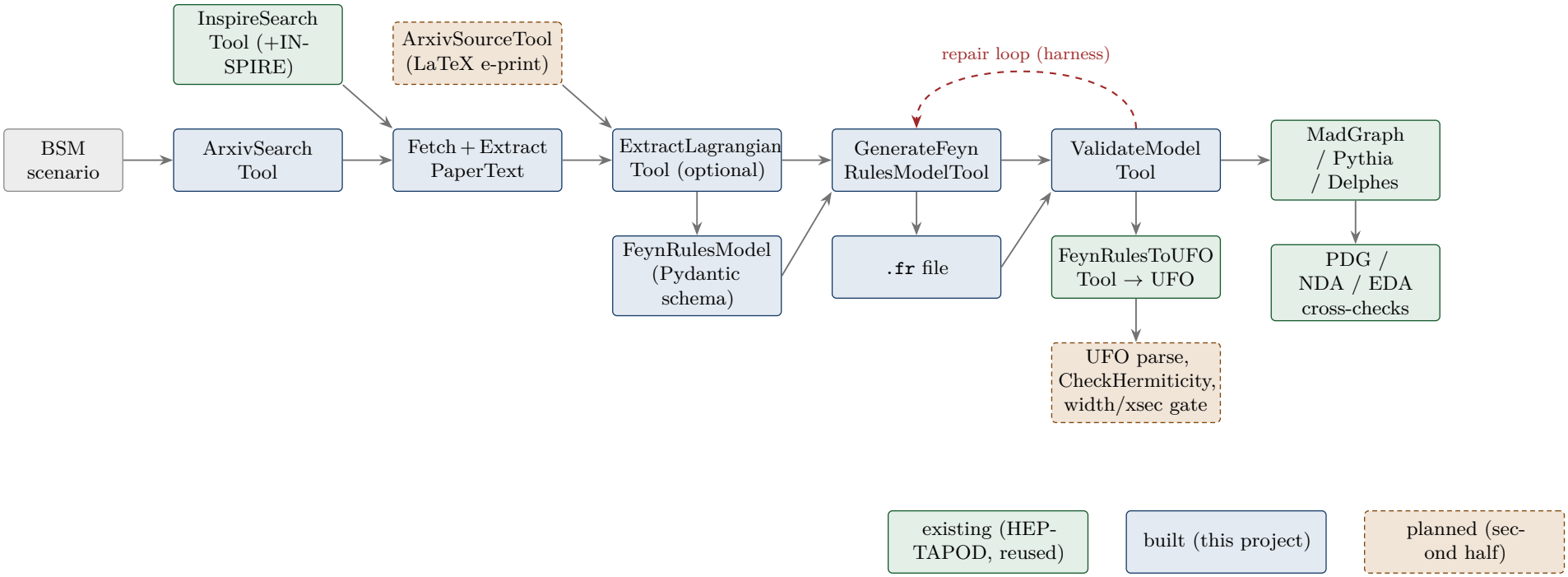

Figure 2: End-to-end workflow. **Green** = existing HEPTAPOD tools we reuse; **blue** = built and tested this term; **orange, dashed** = planned. The dashed red *repair loop* is executed by the driving harness (Figure 1), not by custom code.

3.3 Components: what exists vs. what we build on top

Stage	Component	Status	Role / key technology
Harness	Claude Code / Codex / Orchestral	existing	Swappable driver; owns orchestration + repair loop.
Discovery	`InspireSearchTool` (+10 INSPIRE tools)	existing	Citation-ranked metadata search; reused unchanged.
Discovery	`ArxivSearchTool`	built	arXiv Atom search (ids, PDF links, abstracts).
Ingestion	`FetchPaperPDFTool`, `ExtractPaperTextTool`	built	Sandboxed PDF download (SSRF-guarded) + PyMuPDF full-text.
Ingestion	`ArxivSourceTool` (LaTeX)	planned	Primary extraction input from arXiv `.tex` e-prints (PDF math is lossy).
Extraction	`ExtractLagrangianTool`	built (opt.)	Schema-constrained decode → `FeynRulesModel`; optional—a harness can extract directly.
Schema	`FeynRulesModel` (Pydantic)	built	Central artifact: particles, quantum numbers, parameters, indices; numbers-as-strings preserve rationals ($-1/3$); Lagrangian terms verbatim.
Generation	`GenerateFeynRulesModelTool`	built	Deterministic Jinja2 → syntactically valid `.fr`; round-trips `S1_LQ_RR.fr`.
Validation	`ValidateModelTool`	built	Drives compile + structural checks; returns structured errors for the repair loop.
Validation	`FeynRulesToUFOTool`	existing	Mathematica/FeynRules `.fr`→UFO—the deterministic verifier; reused.
Validation	real UFO parse + `CheckHermiticity` / spectrum	planned	Gauge-invariance / symmetry checks surfaced as named pass/fail.
Validation	analytic width / cross-section gate	planned	Reproduce known Γ, LO σ vs. published values (tolerance).
Downstream	MadGraph / Pythia / Sherpa / Delphes	existing	Event generation and detector simulation; reused.
Cross-checks	PDG, NDA, EDA (Diagrammatica)	existing	Masses/widths, dimensional-analysis sanity checks; reused.
Provenance	Audit trail (prompt) → `audit.json` tool	built/planned	Records sources, extracted terms, conventions, decisions.
Benchmark	`eval` (scoring + runner)	built/planned	Extraction recall / charge accuracy now; diff-vs-reference-`.fr` planned.

3.4 Data flow and design decisions

The `FeynRulesModel` schema is the pipeline's spine: extraction *writes* it, generation *reads* it, validation *checks* the compiled result against it.

- **Structure the error-prone blocks; carry the rest verbatim.** The schema captures particles, quantum numbers, parameters, and index/gauge declarations, numbers stored as

strings so $-1/3$ survives exactly; Lagrangian terms are verbatim Mathematica, with escape hatches for irregular constructs.

- **Deterministic generation.** Jinja2 renders the `.fr`; the LLM never hand-writes Mathematica syntax—removing a class of errors that a "let the harness write the `.fr`" design would reintroduce.
- **Schema-constrained extraction.** Output is well-formed or a repairable validation error is returned.
- **A real verifier and a harness-run repair loop** (Figure 1).
- **Harness- and model-agnostic, auditable**—the same MCP tools run under Claude Code, Codex, or a local Ollama model, with a provenance trail.

4 Novelty and positioning

The *back half* is not novel: ColliderAgent (arXiv:2603.14553) already automates LaTeX-Lagrangian → FeynRules → UFO → MadGraph, and MadAgents (arXiv:2601.21015, v3) runs simulation campaigns from a *user-supplied* PDF. Our contribution is the **front half**: model-name-driven literature *discovery* (INSPIRE/arXiv) and extraction of the Lagrangian *and its conventions* from retrieved paper text with per-term provenance—which neither does. A second differentiator is architectural: ColliderAgent ships as Claude-Code-specific *skills*, whereas our capability is a set of HEPTAPOD *MCP tools* usable from any harness *and* from a reproducible headless loop—the harness-agnostic design of Section 3.1. Related 2026 work, differentiated: `bsm_agent` (arXiv:2606.21316) constructs Lagrangians from user-specified quantum numbers (de-novo, not from literature); FERMIACC / "Albert" generate theories from *data* (SARAH, not FeynRules); Denario (arXiv:2510.26887, linked by the mentors) is a general research-paper agent whose modular design informs our audit-trail and search agents. The field is moving fast (three overlapping agentic-HEP systems in H1 2026), so extraction-with-provenance plus a harness-agnostic tool layer is the durable differentiator.

5 Design revisions from an adversarial self-review

- **Ingest arXiv LaTeX source, not PDF text.** PDF math extraction is measurably weak (Nougat/olmOCR rank formulas the worst modality; plain PyMuPDF is worse). A `.tex` e-print ingestion tool (∼1 week) becomes the primary input, PDF as fallback—also a differentiator (MadAgents still uses PDFs).
- **Re-scope 2HDM as "assisted".** The reference 2HDM is the general Higgs-basis model (matrix Yukawas, basis freedom, restriction files). Target the restricted CP-conserving type-II 2HDM ($\tan\beta$, $\sin(\beta-\alpha)$) as assisted, and keep **scalar leptoquark** and **scalar-singlet dark matter** as the two fully-automatic targets (satisfying "≥2 models").
- **Analytic decay-width reproduction is the primary physics gate.** Closed forms exist and are license-cheap: $\Gamma(S_1 \to q\,\ell) = |y|^2 m_{\mathrm{LQ}}/16\pi$ (arXiv:1801.07641) and $\Gamma(h \to SS)$ (arXiv:1306.4710, whose factor-of-two erratum is a designed test). Cross sections are a secondary LO-vs-LO gate with pinned PDF/scale/seed.
- **Plan around the Mathematica/FeynRules license.** FeynRules compilation and its gauge checks require a license and cannot run in public CI. Split validation: structural checks and comparisons against *published* UFO models run license-free in CI, while FeynRules compilation runs on mentor/Fermilab infrastructure.

- **Deepen validation.** Replace the name-substring UFO check with real `particles.py` parsing (name/PDG/charge/spin/color) and add FeynRules `CheckHermiticity` / `CheckDiagonalKineticTerms` / `CheckMassSpectrum` as named checks.

6 Remaining plan (second half)

- Ship the convention catalog (parse the FeynRules DB `.fr` files).
- Add the arXiv LaTeX-source extraction tool; run the pipeline end-to-end on the scalar leptoquark under Claude Code / Codex and commit the artifacts (audit, model JSON, `.fr`, UFO log).
- Deepen `ValidateModelTool` (real UFO parsing + FeynRules symmetry checks) and add an analytic-width comparison tool.
- Rebuild the benchmark to diff against reference DB implementations (SLQrules, DMsimp, restricted 2HDM); report per-term extraction precision/recall as a headline metric—no prior work measures this task.
- Rebase onto upstream HEPTAPOD v2.2.0 (new `toolkit.yaml` schema) and open a pull request; documentation and worked examples.

7 Evaluation

- **Extraction accuracy:** per-term precision/recall and quantum-number (charge) accuracy vs. reference FeynRules implementations.
- **End-to-end validation pass rate:** fraction of benchmark models whose `.fr` compiles to a UFO *and* reproduces a known width/cross section within tolerance.
- **Reproducibility:** pinned-model reruns give a stable tool trace and `.fr`; a local open model reproduces the workflow (no vendor lock-in).

8 Risks

Extraction accuracy on real hep-ph prose (sign/normalization/chirality conventions) is unmeasured—so measuring it is itself a contribution, with slack budgeted. Sign/normalization errors can pass structural checks and only surface at the width gate, so repair-loop convergence is the key unknown. The Mathematica license gates the physics-validation loop (mitigated by the CI split). Competitive convergence is real; the discovery+provenance and harness-agnostic angles are the hedge.

9 Reproducibility

Code lives on a HEPTAPOD fork (branch `feature/lagrangian-extraction`), registered in `toolkit.yaml`/`test_runner.py`; five offline test suites pass (live LLM/FeynRules paths gated). Setup and a live-demo runbook are in `LAGRANGIAN_EXTRACTION_SETUP.md` and `DEMO.md`. The MCP tool layer is driven from Claude Code / Codex (`claude mcp add` / `codex mcp add`) or the Orchestral web UI—the same tools, a swappable harness.